

Conception et implémentation d'une application multitâche



Nous ne cherchons pas à faire dans ce projet une application temps réel. Les APIs que vous allez manipuler, les phénomènes que vous allez observer et la conception que vous allez mettre en œuvre dans ce projet seraient reproductibles sur un système d'exploitation dit «Temps Réel».

Objectifs

Les objectifs de ce projet sont multiples :

- Concevoir des applications multitâche (non temps réel) avec une approche dirigée par les événements
- Manipuler les mécanismes POSIX d'un système d'exploitation Linux
- Manipuler les mécanismes C11
- Concevoir une application multitâche avec une approche dirigée par le temps

Consignes

Vous devez rendre un compte rendu rédigé en l'intégrant dans une archive .zip accompagnée de vos codes sources et uniquement de vos codes sources. Les binaires seront au préalable supprimés en utilisant la commande `make clean`.

Ce projet zip sera à envoyer par email à l'adresse fthomas.fr@gmail.com.

Un accusé de réception vous sera alors retourné après vérification de l'ouverture de l'archive .zip.

L'ensemble des informations et codes d'exemples concernant la programmation multitâche sont disponibles dans votre cours.

Exercice-1: MultitaskingAccumulator

Comme le montre le schéma d'architecture système de la figure 1, nous cherchons à développer un accumulateur nommé `MultitaskingAccumulator` qui réalise l'acquisition de quatre entrées numériques asynchrones auprès d'un composant logiciel externe nommé `SensorManager` et qui somme ces entrées. Il produit une sortie numérique correspondante à cette somme et une sortie de diagnostic. Ces sorties sont connectées à un `Display` pour affichage.

Le `SensorManager` met à disposition pour chaque entrée un tableau de 256 entiers et un checksum modélisés par le type de donnée `MSG_BLOCK`. La sortie numérique est caractérisée par un tableau de 256 entiers et un checksum modélisés par le même type de donnée `MSG_BLOCK`. La sortie diagnostic est une chaîne de caractères à destination du terminal.

La fonction `getInput` de `SensorManager` permet de simuler la production d'une entrée. La fonction `messageDisplay` de `Display` permet d'afficher le message. De même, la fonction `print` permet d'afficher le nombre de messages produits, consommés et la différence entre les deux.

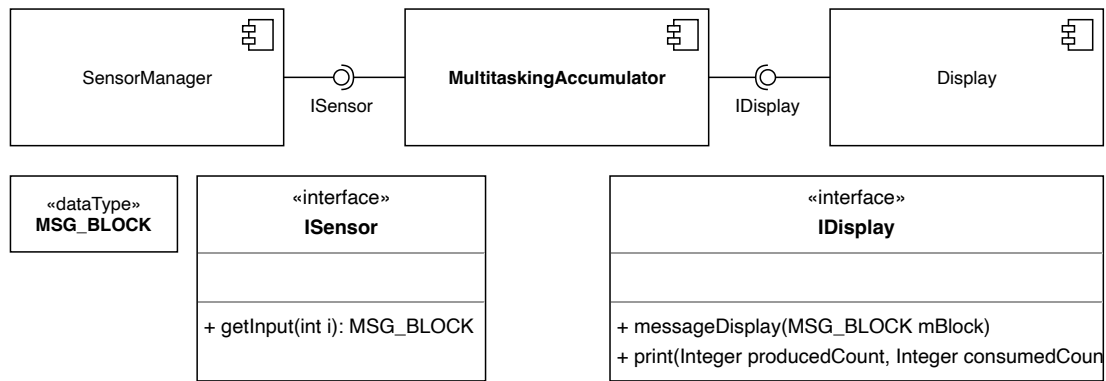


Figure 1: **Architecture Système**. La description de l'architecture système dans laquelle notre logiciel est intégré.

Depuis cette architecture système les exigences suivantes ont été assignées à **MultitaskingAccumulator** :

- **Exigence 1** : **MultitaskingAccumulator** doit produire sur sa sortie **IDisplay** (`messageDisplay`) le cumul de quatre **ISensor** dès qu'une entrée est acquise sur **ISensor**. Le cumul correspond à la production d'un tableau de 256 entiers dont le ième élément de ce tableau est la somme des ièmes éléments des tableaux d'entrée.
- **Exigence 2** : **MultitaskingAccumulator** doit acquérir toutes les données fournies sur son entrée **ISensor**.
- **Exigence 3** : **MultitaskingAccumulator** doit garantir que les données d'entrée de **ISensor** sont correctement formées avant de les sommer pour produire le cumul sur **IDisplay**, sinon il doit supprimer la donnée acquise et relever un message d'erreur dans ses logs produits sur sa sortie standard.
- **Exigence 4** : **MultitaskingAccumulator** doit sommer et produire une sortie sur **IDisplay** au plus vite, c'est-à-dire dès qu'une entrée est présente sur **ISensor** sans attendre une nouvelle valeur sur chacune des entrées.
- **Exigence 5** : **MultitaskingAccumulator** doit produire un diagnostic sur sa sortie **IDisplay** (`print`) de manière asynchrone vis à vis du cumul. Ce diagnostic explicite combien d'entrées ont été acquises, combien ont été sommées et combien restent à sommer. Ces métriques doivent être cohérentes avec le cumul produit sur **IDisplay** (`messageDisplay`). Il n'est pas exigé qu'un diagnostic soit généré pour chaque cumul.

Note: Pour l'exigence 1, nous cherchons à développer un accumulateur. Il est donc inutile d'attendre les quatre entrées pour produire une somme. Au fur et à mesure des valeurs en entrée, le logiciel doit sommer et cela indépendamment de l'entrée. Nous devons sommer deux valeurs successives de la première entrée si elles arrivent avant une valeur sur la deuxième entrée par exemple.

Un architecte logiciel a décrit dans un Software Architecture Document (SAD), l'architecture de **MultitaskingAccumulator** à mettre en œuvre :

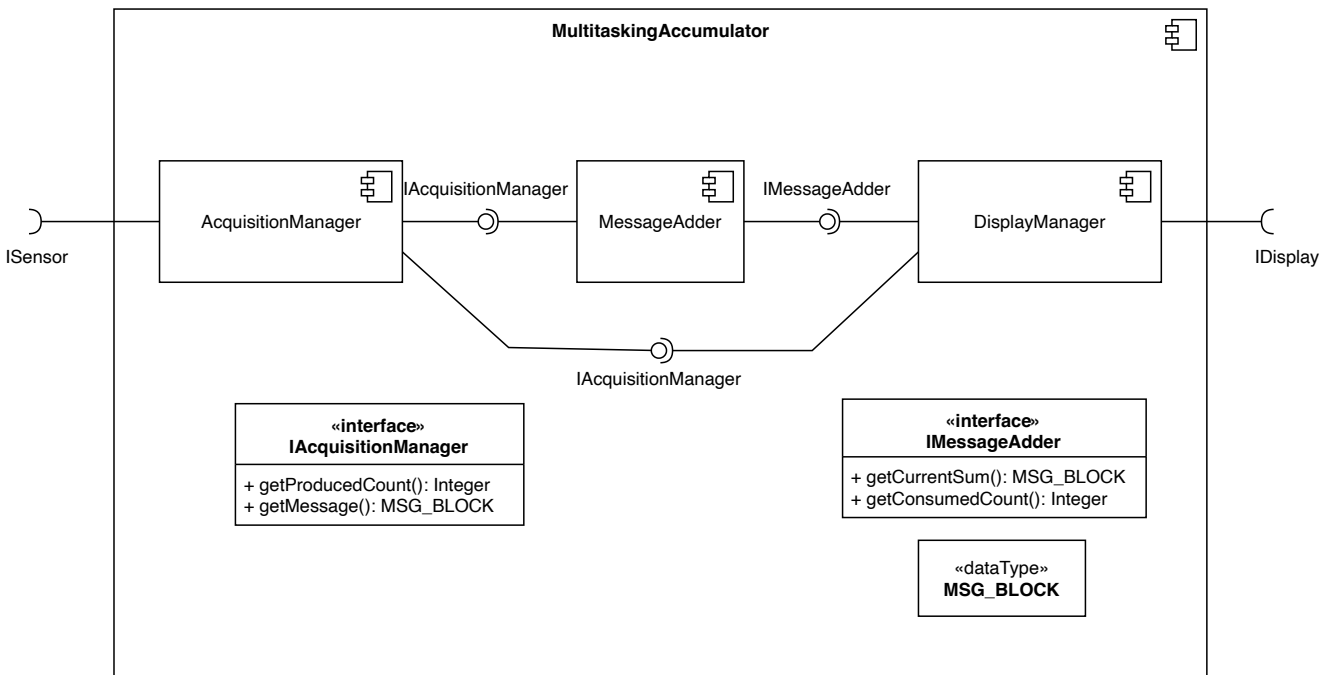


Figure 2: Architecture Logicielle de MultitaskingAccumulator. La description de l'architecture logicielle à mettre en oeuvre.

AcquisitionManager acquiert les entrées depuis l'interface ISensor. MessageAdder somme ces entrées. DisplayManager pilote la sortie. MessageAdder obtient un nouveau message produit par la méthode getMessage d'AcquisitionManager. MessageAdder et DisplayManager peuvent obtenir le nombre de messages produits par la méthode getProducedMessage. De même, DisplayManager obtient le nombre de messages consommés et la somme courante par les méthodes getConsumedCount et getCurrentSum fournies par MessageAdder.

1. Complétez cette architecture logicielle en allouant les exigences de la spécification précédente sur les composants de MultitaskingAccumulator. Un exemple d'allocation à compléter et à modifier est illustré en figure 3. Pour cela, les sources de ce diagramme à compléter sont rangées dans le fichier `diagrams/conception.drawio` editable avec le programme en ligne draw.io (Importez ce fichier dans l'éditeur en ligne pour gagner du temps). (0,5 POINT)

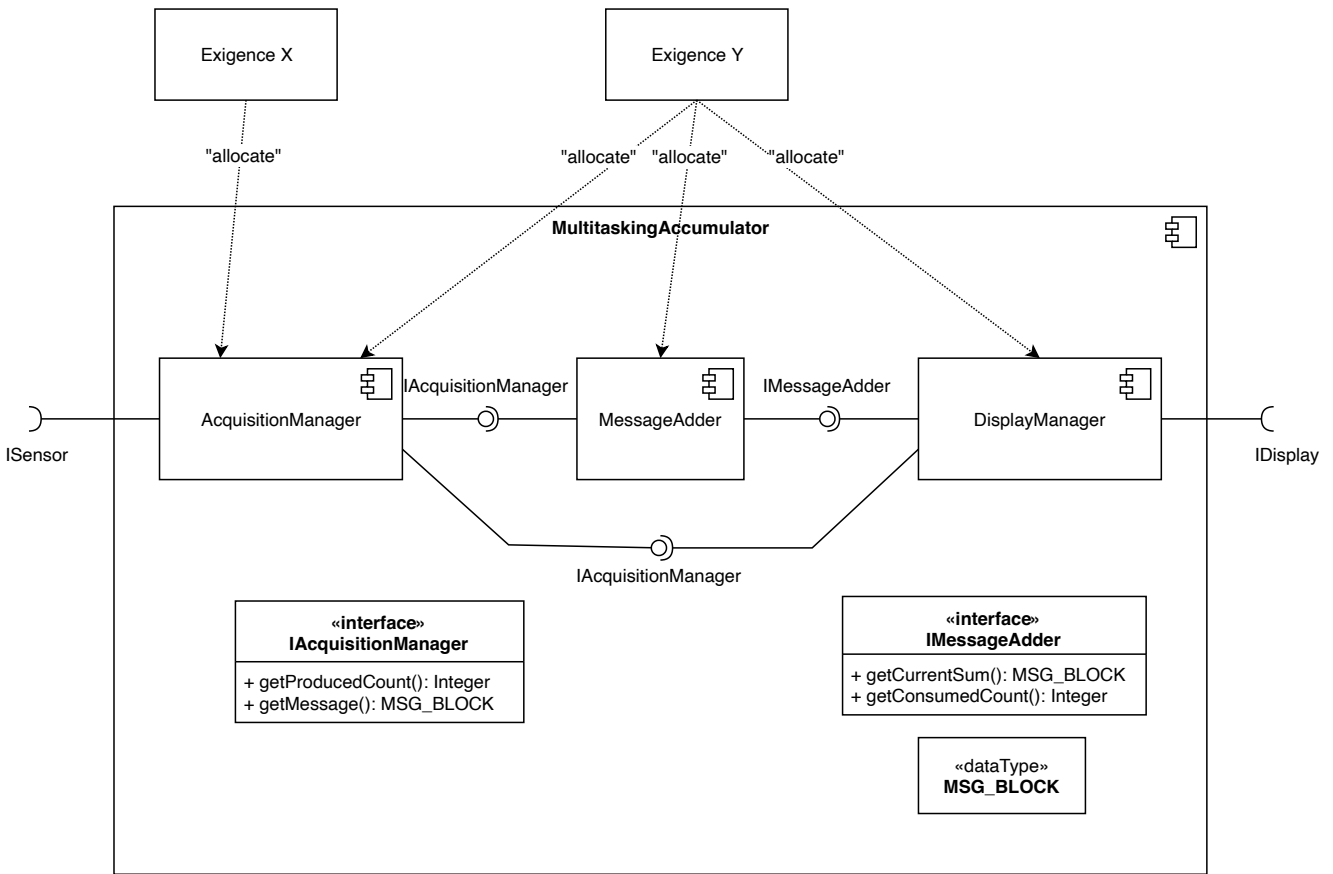


Figure 3: Architecture Logicielle avec allocation des exigences. Un exemple d'allocation d'exigence sur les composants logiciels à modifier/compléter.

L'objectif de ce TP est de décrire l'architecture détaillée multitâche et de proposer plusieurs implémentations.

Conception Détaillée en utilisant une approche dirigée par les événements

2. Une approche dirigée par les événements est-elle une approche synchrone ou asynchrone ? (0,5 POINT)

Plusieurs conceptions détaillées peuvent exister. Nous proposons de guider votre conception en suivant les choix de conception suivants:

- Utilisation de tâches où la mémoire est partagée entre les tâches;
- Appel de la fonction `getInput` dans quatre tâches différentes (une pour chaque entrée) pour aller chercher les données dans `ISensor`. Il n'y a pas d'événement dans `ISensor` indiquant la présence d'une nouvelle entrée. Dès qu'elle est prête la tâche appelle `getInput`.
- Nous n'attendons pas toutes les entrées pour sommer, dès qu'une entrée est disponible, elle est mise dans un tampon en attente d'être sommée. `MessageAdder` cumule cette entrée avec la somme précédente.
- Partage d'un seul tampon de messages `buffer` de taille fixe (taille que vous devez définir) pour échanger les données entre `AcquisitionManager` et `MessageAdder`.
- Nous réduisons au maximum les sections critiques pour fluidifier le programme. Plusieurs écrivains doivent pouvoir écrire en parallèle dans le tampon typiquement.
- Utilisation d'événements sans transmission de données pour synchroniser les tâches;
- Utilisation d'un checksum pour satisfaire l'exigence 3;
- L'utilisation des bibliothèques de messages simulant les entrées (`SensorManager`) et l'affichage (`Display`). La fonction `getInput` permet de simuler la production d'une entrée. La fonction `messageDisplay` permet d'afficher le message. La fonction `print` permet d'afficher le nombre de messages produits, consommés et la différence entre les deux.
- L'utilisation du module `msg.h/msg.c` qui fournit des fonctions utilitaires pour sommer deux messages et vérifier le checksum d'un message.

Nous proposons une architecture détaillée préliminaire en figure 2 et un squelette d'implémentation de cette architecture dans les `.h` et `.c` fournis. Etudiez le squelette d'implémentation `.h` et `.c` fournis et l'architecture logicielle détaillée ci-dessous pour en comprendre les structures de données et les APIs.

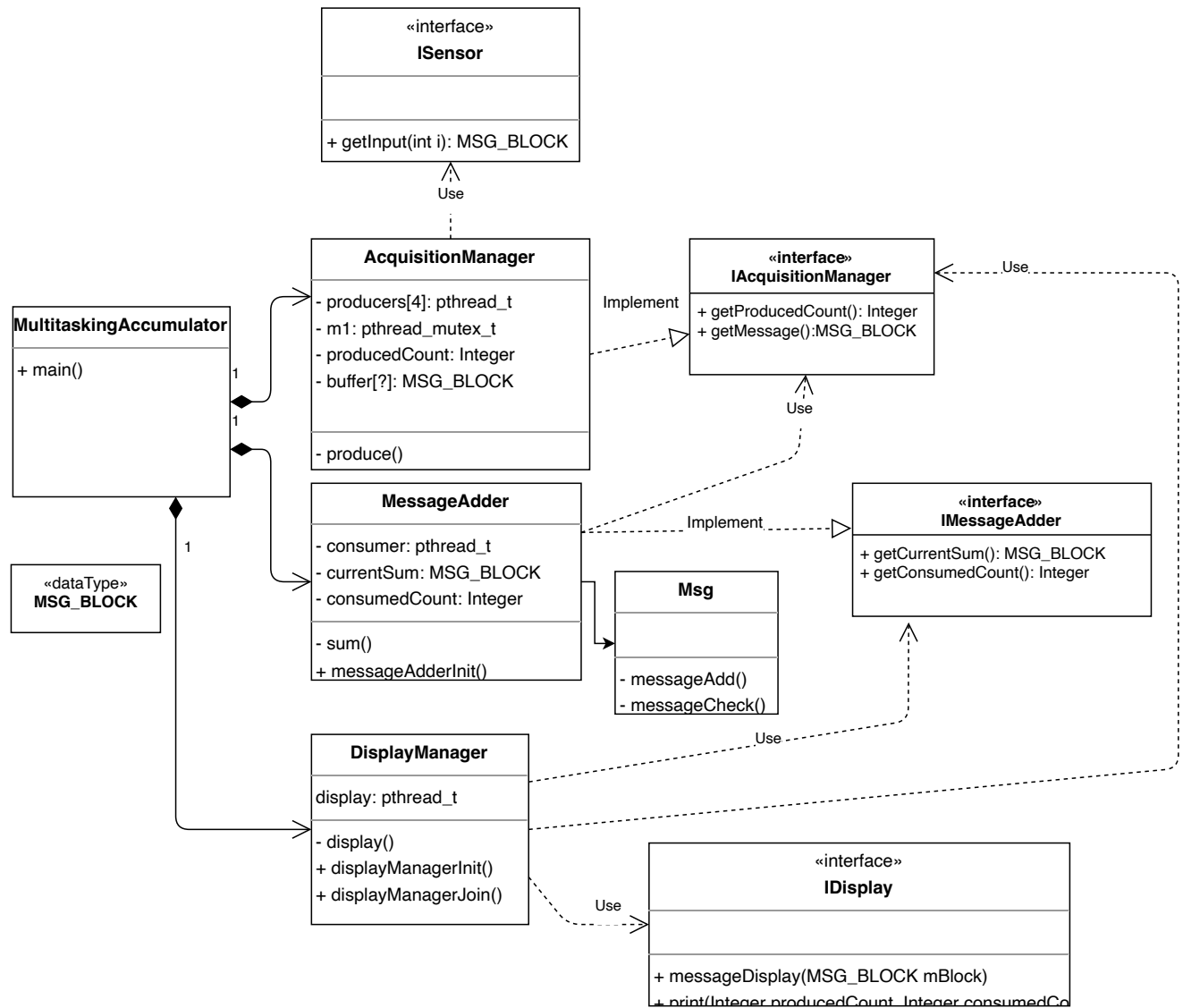


Figure 4: Architecture Logicielle Détaillée de MultitaskingAccumulator. La description de l'architecture logicielle détaillée mise en oeuvre.

3. A quelle stratégie de sûreté la méthode `messageCheck` répond-t-elle ? (1 POINT)
4. A ce stade de l'implémentation squelette proposée, combien y a-t-il de processus (`process`) et de fil d'exécution (`thread`) POSIX dans ce programme ? (0,5 POINT)
5. Comment est calculé le checksum d'un message dans la fonction `messageCheck` ? (1 POINT)
6. Complétez cette architecture logicielle détaillée par l'analyse des tâches à mettre en oeuvre, des échanges de données et des synchronisations entre les tâches en satisfaisant les choix de conception précédents. (3 POINTS)

Cette conception devra donc détailler l'architecture statique et dynamique. Vous devrez mettre à jour le diagramme de classes pour y ajouter les APIs dont vous avez besoin et que vous utilisez dans le diagramme de séquence. Vous devrez fournir dans votre rapport ce diagramme de classes et un diagramme de séquence.

Le diagramme de séquences est utilisé pour expliquer et démontrer la causalité des traitements. Un exemple de diagramme de séquence est illustré ci-dessous. Vous pouvez repartir de cet exemple en utilisant le formalisme [plantuml](#) disponible en ligne pour réaliser de manière automatique vos [diagrammes de séquences UML](#). Un éditeur [plantuml](#) est utilisable directement en ligne [plantText](#). Copiez/collez le code présent dans le fichier `diagrams/preamble.puml` dans l'éditeur en ligne puis cliquez sur `Refresh`. Cela doit vous donner le diagramme ci-dessous. Le diagramme de classes est soit réalisés à partir de zéro dans [plantText](#) soit complété à partir du fichier `drawio` disponible dans le répertoire `diagrams/conception.drawio`.

Vous indiquerez dans votre conception les valeurs initiales des sémaphores si vous utilisez des sémaphores.

Auriez-vous pu utiliser les mêmes choix de conception que ceux-ci-dessus ? Quels mécanismes auriez-vous mis en oeuvre ? (1 POINT)

Nous cherchons désormais à optimiser notre programme.

11. Proposez une solution pour protéger de manière efficace et sans utiliser des apis POSIX, le compteur permettant compter le nombre de messages produits ? (1 POINT)

12. Implémentez cette solution dans la méthode `incrementProducerCount` et `getProducerCount` dans le fichier `acquisitionManagerAtomic.c` et montrez un résultat d'exécution. (1 POINT)

Pour compiler votre programme, deux règles dans le makefile est proposée :

- `make atomic`: compile le programme incluant le fichier `acquisitionManagerAtomic.c`
- `make runatomic`: compile le programme incluant le fichier `acquisitionManagerAtomic.c`

13. Proposez une deuxième solution pour que ces méthodes `incrementProducerCount` et `getProducerCount` en vous basant sur la méthode `atomic_compare_exchange_weak` ? (1 POINT)

14. Implémentez cette solution dans la méthode `incrementProducerCount` et `getProducerCount` en définissant deux méthodes `pCountLockTake()` et `pCountLockRelease()`. Montrez un résultat d'exécution. (1 POINT)

- `make testandset`: compile le programme incluant le fichier `acquisitionTestAndSet.c` utilisé en question 13
- `make runtestandset`: compile le programme incluant le fichier `acquisitionTestAndSet.c` utilisé en question 13

Nous avons exécuté plusieurs fois chacun des programmes précédents (`MultitaskingAccumulatorPosix` utilisant les mutex POSIX, `MultitaskingAccumulatorAtomic` utilisant une implémentation C11 Atomic et `MultitaskingAccumulatorTestA` utilisant la méthode C11 `atomic_compare_exchange_weak`). Nous avons mesuré le nombre d'appel à `getProducerCount`, son temps total d'exécution et son temps moyens d'exécution. Les résultats sont présentés ci-dessous.

Process	NbCalls	TotalExecTime	AvgExecTime
MultitaskingAccumulatorPosix[8193]	8 times,	866us total,	108us avg
MultitaskingAccumulatorPosix[8247]	8 times,	199us total,	24us avg
MultitaskingAccumulatorPosix[8215]	8 times,	393us total,	49us avg
MultitaskingAccumulatorPosix[8270]	8 times,	235us total,	29us avg
MultitaskingAccumulatorPosix[8143]	8 times,	163us total,	20us avg
MultitaskingAccumulatorPosix[8239]	8 times,	646us total,	80us avg
MultitaskingAccumulatorPosix[8262]	8 times,	149us total,	18us avg
MultitaskingAccumulatorPosix[8159]	8 times,	344us total,	43us avg
MultitaskingAccumulatorPosix[8176]	8 times,	498us total,	62us avg
MultitaskingAccumulatorPosix[8278]	8 times,	182us total,	22us avg
MultitaskingAccumulatorAtomi[9092]	8 times,	123us total,	15us avg
MultitaskingAccumulatorAtomi[9116]	8 times,	124us total,	15us avg
MultitaskingAccumulatorAtomi[9100]	8 times,	278us total,	34us avg
MultitaskingAccumulatorAtomi[8905]	8 times,	144us total,	18us avg
MultitaskingAccumulatorAtomi[8819]	8 times,	116us total,	14us avg
MultitaskingAccumulatorAtomi[8897]	8 times,	169us total,	21us avg
MultitaskingAccumulatorAtomi[9083]	8 times,	113us total,	14us avg
MultitaskingAccumulatorAtomi[8889]	8 times,	116us total,	14us avg
MultitaskingAccumulatorAtomi[9108]	8 times,	180us total,	22us avg
MultitaskingAccumulatorAtomi[9067]	8 times,	118us total,	14us avg
MultitaskingAccumulatorTestA[9783]	8 times,	377us total,	47us avg
MultitaskingAccumulatorTestA[9732]	8 times,	234us total,	29us avg
MultitaskingAccumulatorTestA[9757]	8 times,	226us total,	28us avg
MultitaskingAccumulatorTestA[9723]	8 times,	715us total,	89us avg
MultitaskingAccumulatorTestA[9748]	8 times,	131us total,	16us avg
MultitaskingAccumulatorTestA[9773]	8 times,	148us total,	18us avg
MultitaskingAccumulatorTestA[9793]	8 times,	878us total,	109us avg
MultitaskingAccumulatorTestA[9740]	8 times,	635us total,	79us avg
MultitaskingAccumulatorTestA[9801]	8 times,	541us total,	67us avg
MultitaskingAccumulatorTestA[9765]	8 times,	167us total,	20us avg

15. Concluez sur les différentes mesures et identifiez une des causes dès que l'on utilise des APIs POSIX (1 POINT)

Conception détaillée en utilisant une approche dirigée par le temps

16. Une approche dirigée par le temps est-elle une approche synchrone ou asynchrone ? (0,5 POINT)

Nous souhaitons ajouter des exigences liées au temps et définir une conception utilisant le modèle Psy vue en cours (Logical Execution Model). Les exigences supplémentaires sont les suivantes:

- **Exigence 6 :** Le logiciel doit acquérir les entrées toutes les 100 ms.
- **Exigence 7 :** Le délai de bout en bout entre l'acquisition d'une entrée et la sortie affichée incluant cette entrée est de maximum 600 ms.

17. En reprenant la représentation par ligne de temps et par messages illustrée dans la figure suivante, proposez une conception dirigée par le temps simplifiée de ce même programme MultitaskingAccumulator. (1 POINT)

Une première ligne horizontale sera celle de l'horloge. Les ticks d'horloge seront des points. Ensuite, une ligne par tâche sera représentée. Les points de synchronisation temporels sur ces tâches seront représentés par des lignes verticales (les instructions `advance` en PsyC).

La représentation des fonctions élémentaires entre les points de synchronisation temporels seront représentés par des rectangles (il n'est pas nécessaire de détailler le nom des fonctions et leurs nombres). Les messages produits entre les tâches seront représentés par des enveloppes à l'instant ou ces messages sont visibles des consommateurs (voir exemple ci-dessous et celui vu en cours en fin de partie 2).

Les sources de ce diagramme à compléter sont rangées dans le fichier `diagrams/conception.drawio` editable avec le programme en ligne draw.io (Importez ce fichier dans l'éditeur en ligne pour gagner du temps).

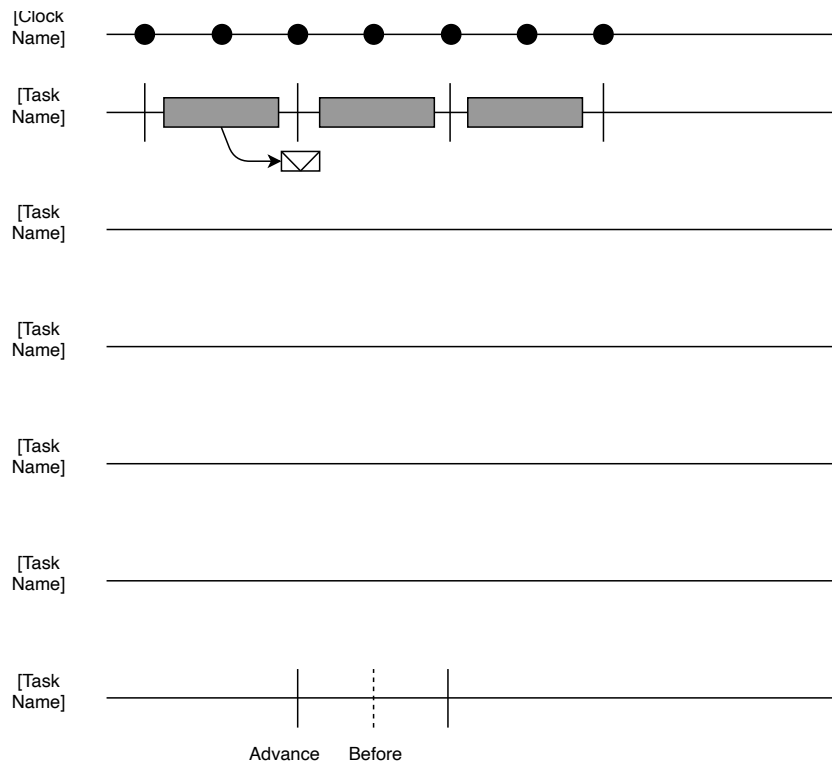


Figure 6: Diagramme de temps de MultitaskingAccumulator dans une approche dirigée par le temps. Description d'une conception dirigée par le temps à compléter/modifier.

18. Comment avez-vous géré le jitter de sortie ? (1 POINT)

Pour gérer le jitter, vous pouvez représenter les instructions `before` en PsyC en représentant des lignes verticales hachurées. Cette instruction vous garantit que la fonction élémentaire sera exécutée avant ce point dans le temps même si la prochaine exécution ne débutera qu'après le prochain point de synchronisation temporel.